

Taxonomy Report

López-Chablé Edgard Uriel

Contents

1. Data Import and Preprocessing	1
2. Global Species Abundance	2
3. Lowest Common Ancestor (LCA) Read Distribution	3
Session Information	6

```
# Load necessary libraries
library(ggplot2)
library(dplyr)
library(stringr)
library(plotly)
library(networkD3)
library(htmlwidgets)
library(webshot)
library(magick)
```

1. Data Import and Preprocessing

We import the Kraken2 taxonomy report and perform initial cleanup, such as removing leading/trailing whitespaces from the taxon names.

```
# Create output directory for high-res figures
out_dir <- "../results/taxonomy"
dir.create(out_dir, showWarnings = FALSE, recursive = TRUE)

# Read Kraken2 report
kraken_data <- read.delim("../data/taxonomy/reporte_taxonomia.txt",
                          header = FALSE,
                          sep = "\t",
                          col.names = c("Percentage", "Clade_Reads", "Taxon_Reads",
                                         "Rank", "TaxID", "Name"))

# Clean up whitespace used by Kraken for indentation
kraken_data <- kraken_data %>%
  mutate(Name = str_trim(Name))

head(kraken_data)
```

```
##   Percentage Clade_Reads Taxon_Reads Rank  TaxID      Name
```

## 1	5.61	2368311	2368311	U	0	unclassified
## 2	94.39	39842714	37855	R	1	root
## 3	94.30	39804838	667	R1	131567	cellular organisms
## 4	94.29	39802172	128712	D	2	Bacteria
## 5	93.98	39668637	70770	P	1224	Pseudomonadota
## 6	93.67	39538156	197192	C	1236	Gammaproteobacteria

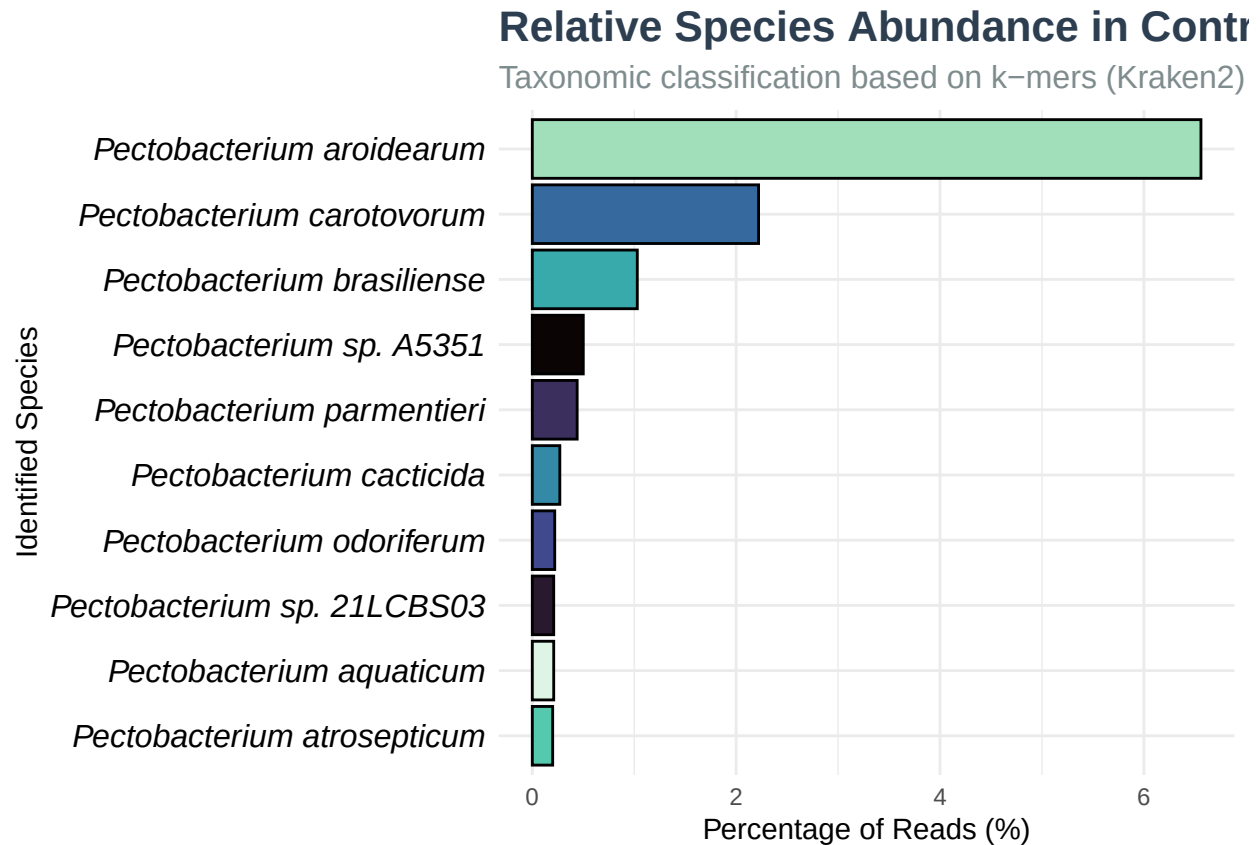
2. Global Species Abundance

We filter the data strictly to the Species level (Rank ‘S’) to visualize the top 10 most abundant species in our sample.

```
# Filter and sort for the top 10 species
bar_data <- kraken_data %>%
  filter(Rank == "S") %>%
  arrange(desc(Percentage)) %>%
  head(10)

# Create the bar plot
p_bar <- ggplot(bar_data, aes(x = reorder(Name, Percentage), y = Percentage, fill = Name)) +
  geom_bar(stat = "identity", color = "black", show.legend = FALSE) +
  coord_flip() +
  theme_minimal() +
  labs(title = "Relative Species Abundance in Control Sample",
       subtitle = "Taxonomic classification based on k-mers (Kraken2)",
       x = "Identified Species",
       y = "Percentage of Reads (%)") +
  theme(
    plot.title = element_text(face = "bold", size = 16, color = "#2c3e50"),
    plot.subtitle = element_text(size = 12, color = "#7f8c8d"),
    axis.text.y = element_text(face = "italic", size = 12, color = "black")
  ) +
  scale_fill_viridis_d(option = "mako", direction = -1)

p_bar
```



```
# Export to PDF and 600 DPI TIFF
```

```
ggsave(filename = paste0(out_dir, "/barplot_taxonomy.pdf"), plot = p_bar, width = 10, height = 8)
ggsave(filename = paste0(out_dir, "/barplot_taxonomy.tiff"), plot = p_bar, width = 10, height = 8, dpi = 600)
```

3. Lowest Common Ancestor (LCA) Read Distribution

Not all reads assigned to the Genus level can be resolved to a specific Species. This Sankey diagram visualizes the flow of reads and highlights the proportion of sequences conserved at the Genus level due to the LCA algorithm.

```
# Extract total reads for the Pectobacterium genus
```

```
genus_reads <- kraken_data %>%
  filter(Rank == "G" & Name == "Pectobacterium") %>%
  pull(Clade_Reads)
```

```
# Extract breakdown of specific Pectobacterium species
```

```
species_data <- kraken_data %>%
  filter(Rank == "S" & grepl("Pectobacterium", Name)) %>%
  select(Name, Clade_Reads)
```

```
# Calculate remaining reads (LCA algorithm resolution limit)
```

```
sum_species <- sum(species_data$Clade_Reads)
```

```

unclassified_reads <- genus_reads - sum_species

# Build Sankey Links
links <- data.frame(
  source = rep("Pectobacterium (Total Genus)", nrow(species_data) + 1),
  target = c(species_data$Name, "Conserved at Genus Level (LCA)"),
  value = c(species_data$Clade_Reads, unclassified_reads)
)

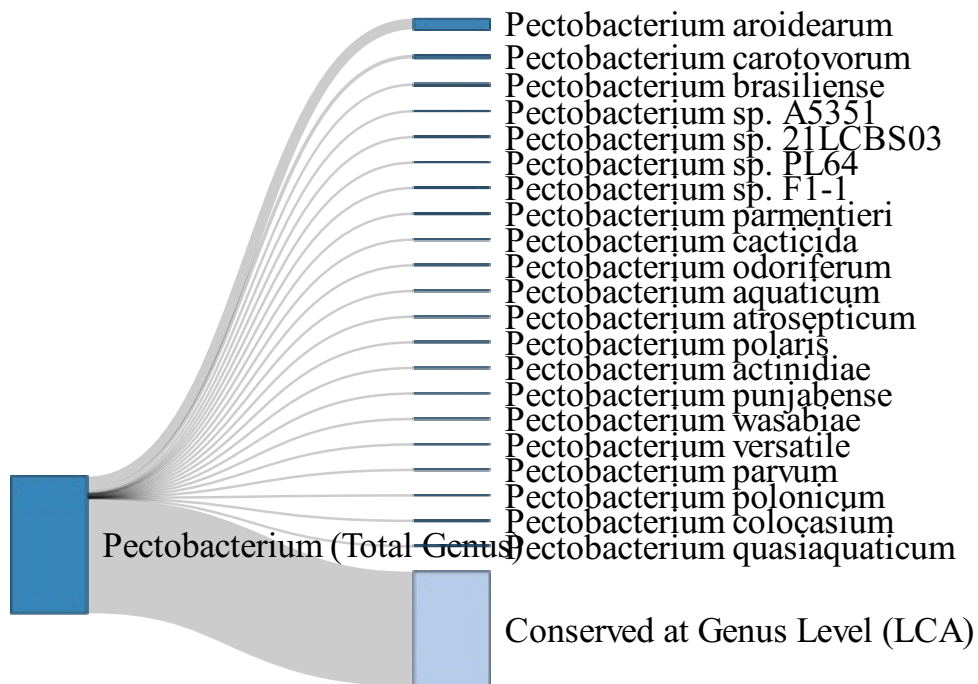
# Build Nodes and assign numerical IDs required by networkD3
nodes <- data.frame(name = unique(c(links$source, links$target)))

links$IDsource <- match(links$source, nodes$name) - 1
links$IDtarget <- match(links$target, nodes$name) - 1

# Draw interactive Sankey Network
sankey_plot <- sankeyNetwork(Links = links, Nodes = nodes,
  Source = "IDsource", Target = "IDtarget",
  Value = "value", NodeID = "name",
  units = " Reads",
  fontSize = 14, nodeWidth = 30,
  sinksRight = FALSE,
  margin = list(left = 50, right = 50))

sankey_plot

```



```
# Export Interactive Sankey to Static High-Res Formats
```

```
# 1. Save as temporary HTML
```

```
saveWidget(sankey_plot, "temp_sankey.html", selfcontained = FALSE)
```

```
## Warning in dir.create(target_dir): 'temp_sankey_files\htmltools-fill-0.5.9'
```

```
## already exists
```

```
## Warning in dir.create(target_dir): 'temp_sankey_files\htmlwidgets-1.6.4'
```

```
## already exists
```

```
## Warning in dir.create(target_dir): 'temp_sankey_files\d3-4.9.1' already exists
```

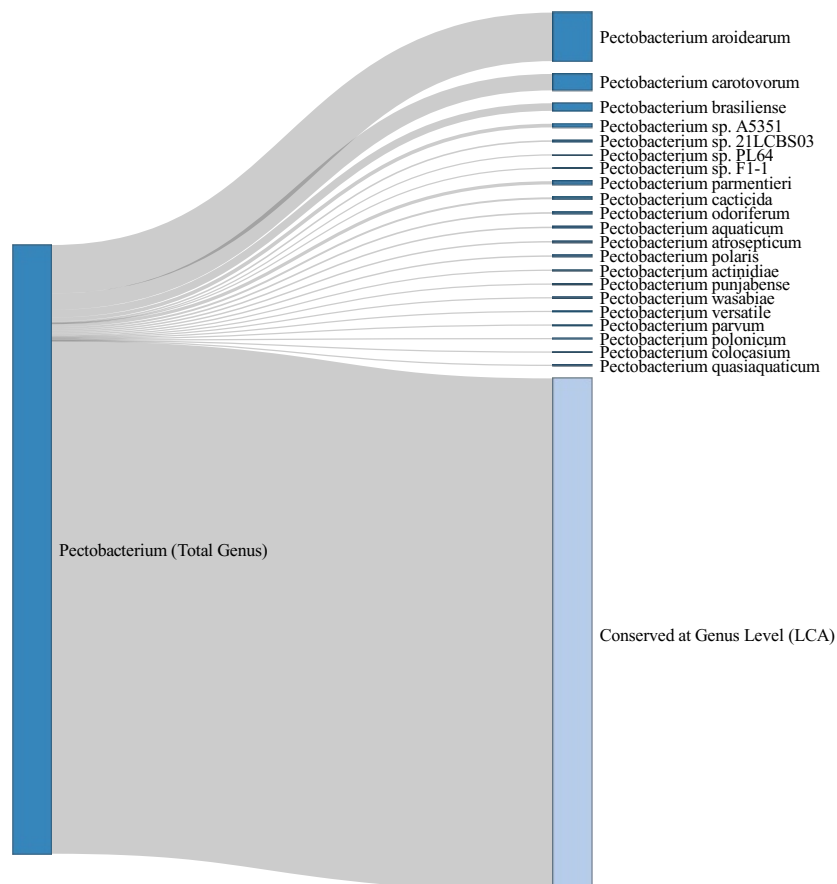
```
## Warning in dir.create(target_dir): 'temp_sankey_files\sankey-1' already exists
```

```
## Warning in dir.create(target_dir):
```

```
## 'temp_sankey_files\sankeyNetwork-binding-0.4.1' already exists
```

```
# 2. Capture as PDF
```

```
webshot("temp_sankey.html", file = paste0(out_dir, "/sankey_lca.pdf"))
```



```

# 3. Read PDF and render as 600 DPI TIFF using magick
img_sankey <- image_read_pdf(paste0(out_dir, "/sankey_lca.pdf"), density = 600)
image_write(img_sankey, paste0(out_dir, "/sankey_lca.tiff"), format = "tiff")

# Clean up temporary HTML files
unlink(c("temp_sunburst.html", "temp_sankey.html", "temp_sunburst_files", "temp_sankey_files"), recursive = TRUE)

```

Session Information

```
sessionInfo()
```

```

## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26120)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
##  [1] LC_COLLATE=Spanish_Mexico.utf8  LC_CTYPE=Spanish_Mexico.utf8
##  [3] LC_MONETARY=Spanish_Mexico.utf8 LC_NUMERIC=C
##  [5] LC_TIME=Spanish_Mexico.utf8
##
## time zone: America/Mexico_City
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] magick_2.9.1      webshot_0.5.5      htmlwidgets_1.6.4  networkD3_0.4.1
## [5] plotly_4.12.0     stringr_1.6.0      dplyr_1.2.1        ggplot2_4.0.3
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4      tidyr_1.3.2        stringi_1.8.7      digest_0.6.39
##  [5] magrittr_2.0.5      evaluate_1.0.5     grid_4.5.2         RColorBrewer_1.1-3
##  [9] fastmap_1.2.0       jsonlite_2.0.0     processx_3.9.0     httr_1.4.8
## [13] purrr_1.2.2         viridisLite_0.4.3  scales_1.4.0       lazyeval_0.2.3
## [17] textshaping_1.0.5   pdftools_3.9.0     cli_3.6.5          rlang_1.2.0
## [21] withr_3.0.2         yaml_2.3.12        otel_0.2.0         tools_4.5.2
## [25] vctrs_0.7.3         R6_2.6.1           lifecycle_1.0.5    ragg_1.5.2
## [29] pkgconfig_2.0.3     callr_3.7.6        pillar_1.11.1      gtable_0.3.6
## [33] data.table_1.18.4   glue_1.8.1         Rcpp_1.1.1-1.1     systemfonts_1.3.2
## [37] xfun_0.56           tibble_3.3.1       tidyselect_1.2.1   data.tree_1.2.0
## [41] rstudioapi_0.18.0   knitr_1.51         farver_2.1.2       htmltools_0.5.9
## [45] igraph_2.3.1        rmarkdown_2.31     labeling_0.4.3     qpdf_1.4.1
## [49] compiler_4.5.2     S7_0.2.2           askpass_1.2.1

```